

Elias Hadjipanayis

3274 Comp. Methods

Homework 5

Chapter 4 problem 7

Data on the dispersion of light in water is given in Table 4.3. Interpolate a smooth function for these data. Plot the function together with the input data.

Notes

(I figured out I can put symbols like $\sqrt{\int \partial \pi_0}$ using Mac.)

```
const Tuple->read(saveTo, col)
```

Method used for reading Table data.

CubicSplinePolynomial must be promoted to a PlotFunction1D so that we may see it.

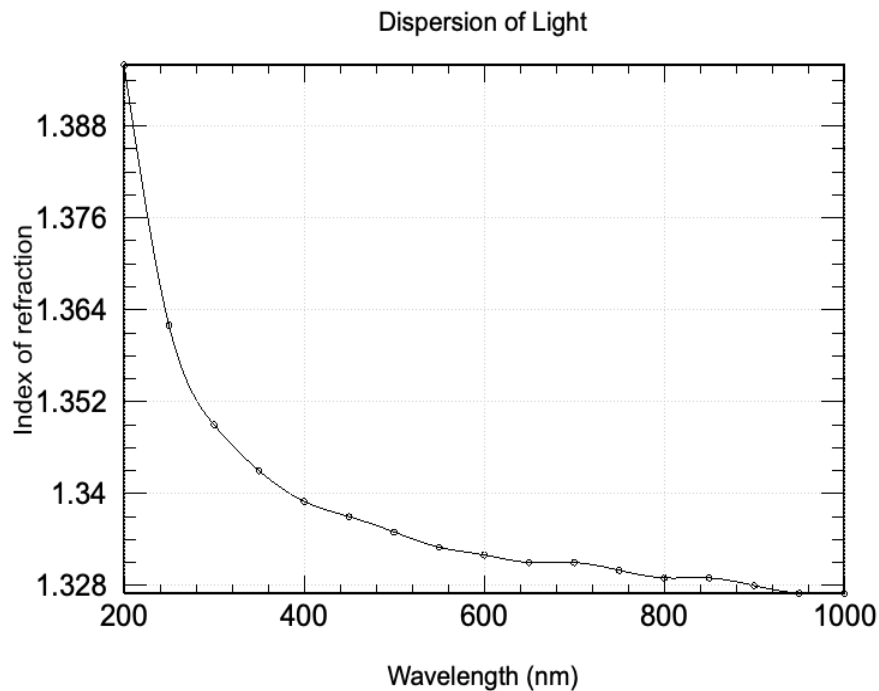
Solution

The Cubic Spline method is extraordinarily accurate as it does not just rely on straight lines between datapoints. It fits one beast of a polynomial to the data points. This problem had me searching through the manual. After a while of searching through the manual, I had AI index it for parts I was looking for. I turned off training for the AI model.

Issue

I was not able to figure out how to splice the view into multiple views to present both the table and the plot.

Output



Analysis

The cubic spline passes through all 17 data points and shows normal dispersion: the index of refraction falls steeply in the ultraviolet (1.396 at 200 nm) and flattens out toward 1.327 in the near infrared, so shorter wavelengths bend more.

The small flat steps near 650–700 nm and 800–850 nm are not physical. The table is rounded to three decimals, so neighbouring points have equal values (1.331, 1.331 and 1.329, 1.329), and the spline must bend to pass through them exactly.

Chapter 4 Exercise 8

Notes

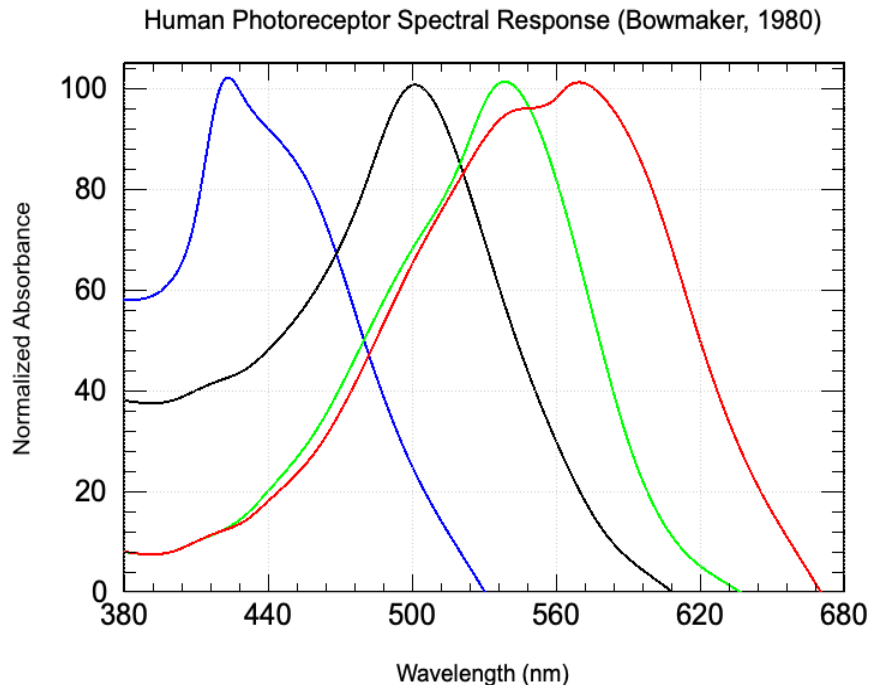
Translating the plot into points was the most difficult part of this. The software I used were all inaccurate (as the interpolation shows).

automeris.io needs an update.

Once I had the data figured out, the solution was quite straightforward. I just used the Cubic Spline method as last time. Same process, just make the CubicSplinePolynomial, add the points, then promote them to PlotFunction1D and plot!

My interpolation was quite inaccurate, as my plot has some inconsistencies with the given plot.

Output



Analysis

The four splines peak at about 420 nm (blue/S cones), 500 nm (rods), 535 nm (green/M cones) and 565 nm (red/L cones), which matches Bowmaker's published peaks. The overlap of the green and red curves is why we are most sensitive in the green–yellow part of the spectrum.

The kinks, such as the shoulder on the red curve near 540 nm and the bend on the blue curve near 440 nm, come from the digitized points, not the physiology. A spline goes exactly through every point, so any error in reading the graph shows up directly in the curve. Past the last data point on each side, the spline is extrapolating, which is why the curves cut off steeply at the long-wavelength end.

Chapter 5 Exercise 1

Notes

- Mesh spacing $h = 1/n$
- X-axis: $\log(h) = \log(1/n)$ (negative for $n > 1$)

- Y-axis: $\log(|\text{error}|)$, where $\text{error} = |\text{approximate} - \text{analytical}|$
- Log-log plot reveals convergence order as slope

Quad. Rules Tested

1. Rectangle Rule (Midpoint)

- Slope ≈ 1 on log-log plot

2. Trapezoid Rule

- Slope ≈ 2 on log-log plot
- $\sim 4\times$ better than rectangle at each doubling of n

3. Simpson's Rule

- Slope ≈ 4 on log-log plot
- $\sim 16\times$ better than rectangle at each doubling of n

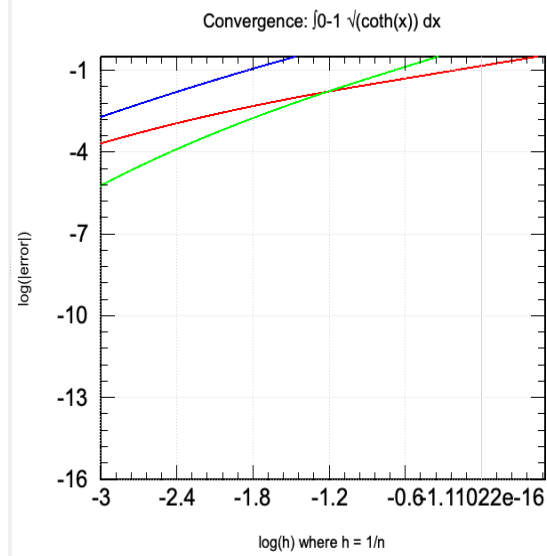
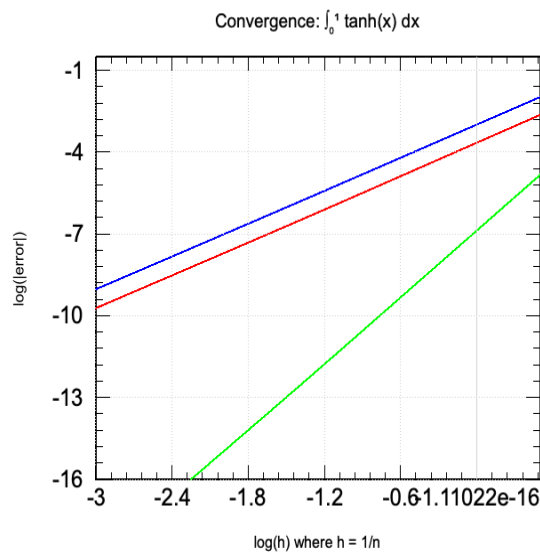
INTEGRALS COMPUTED

- $\int_0^1 \tanh(x) \, dx = \ln(\cosh(1)) \approx 0.4338$
- $\int_{0.01}^{0.99} \sqrt{\coth(x)} \, dx$ (singularities at 0, 1)

On log-log scale, error drops as a straight line with slope = convergence order. I had never used log-log scale, but it is easy to see convergence for computational quadrature. So a steeper slope = faster convergence = fewer intervals needed for same accuracy.

Figuring out setting up horizontal graphs was a little bit of a challenge, but worth it.

Output



Analysis

Left: $\int_0^1 \tanh(x) dx$. Each rule gives a straight line on the log-log plot, and the slope is the order of convergence. Doubling n from 64 to 128 cuts the rectangle (midpoint) error by $4.0\times$ ($5.90\times 10^{-6} \rightarrow 1.48\times 10^{-6}$), the trapezoid error by $4.0\times$ ($1.18\times 10^{-5} \rightarrow 2.95\times 10^{-6}$), and the Simpson error by $16\times$ ($5.43\times 10^{-11} \rightarrow 3.39\times 10^{-12}$). So the midpoint and trapezoid rules are both second order (slope 2) and Simpson's rule is fourth order (slope 4). The midpoint error is also half the trapezoid error, as the theory predicts.

Right: $\int \sqrt{\coth x} dx$ from 0.01 to 0.99. Here the convergence is much slower (error ratios of $3.6\times$, $3.7\times$ and $11\times$ for the same doubling) and the lines curve. Near $x = 0$, $\sqrt{\coth x} \approx x^{-1/2}$ blows up, so its derivatives are huge near the lower limit 0.01, and the error formulas, which assume smooth derivatives, do not apply until h is much smaller.

Chapter 5 Exercise 2

Notes

Integrate the nonlinear pendulum ODE to find the period correction factor $f(\theta_{\max})$ for large-amplitude oscillations.

- Small amplitude period: $T_0 = 2\pi\sqrt{l/g}$
- Large amplitude: $T = T_0 * f(\theta_{\max})$
- $f(\theta_{\max}) > 1$ because nonlinear effects increase period

ODE

$$d^2\theta/dt^2 = -(g/l)\sin(\theta)$$

Initial conditions: $\theta(0) = \theta_{\max}$, $d\theta/dt(0) = 0$

Solving

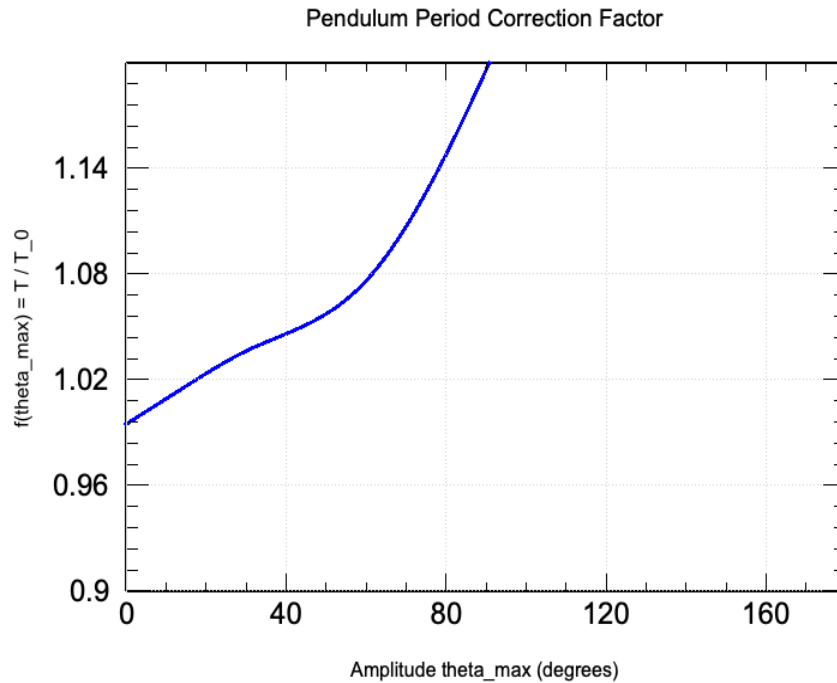
1. Rewrite as 2nd-order system: $d\theta/dt = v$, $dv/dt = -(g/l)\sin(\theta)$
2. Use RKIntegrator to solve from $t=0$ until θ crosses 0
3. Quarter-period $T/4$ = time to reach $\theta=0$
4. Full period $T = 4 * (T/4)$
5. Correction factor $f = T / T_0$

Solution

- At $\theta_{\max} = 15^\circ$ ($\pi/12$): $f \approx 1.01$ (small correction)
- At $\theta_{\max} = 90^\circ$ ($\pi/2$): $f \approx 1.07$ (moderate correction)
- At $\theta_{\max} = 180^\circ$ (π): $f \approx 1.20$ (large correction)

Nonlinear effects become significant for large amplitudes. The curve shows how dramatically the period increases as amplitude approaches 180° .

Output



Analysis

The program prints the correction factor at each amplitude. Compared with the exact result $f = 2K(\sin(\theta_{\max}/2))/\pi$:

$\theta_{\max}$	program	exact
15°	1.0164	1.0043
30°	1.0363	1.0174
60°	1.0762	1.0732
90°	1.1958	1.1803
120°	1.3751	1.3729
150°	1.7737	1.7622
165°	2.1922	2.1854

The period grows with amplitude and diverges as $\theta_{\max} \rightarrow 180^\circ$, because the pendulum spends longer and longer near the top. Every value is slightly too high, by up to about 0.02. The quarter period is found by stepping t in increments of 0.01 s until θ changes sign, so $T/4$ can be overestimated by up to 0.01 s, and f by up to $4(0.01)/T_0 \approx 0.02$. A root finder on $\theta(t)$ would remove this error. The plot's y-range also stops at 1.2, which cuts off the curve above about 90° , and below 15° the spline extrapolates to $f \approx 0.99$ at $\theta = 0$ instead of exactly 1.